



SECURITY AUDIT

1inch

Differential

FINAL REPORT

Aug 2026

BAILSEC.IO



Disclaimer:

BailSec was engaged by the client to perform a time-boxed technical security assessment of the target identified in this report. The assessment applies only to the scope, version, commit, deployment, materials and information identified in the report. Its findings are point-in-time and not exhaustive. Changes made after the assessment—including remediation changes—were not assessed unless expressly stated otherwise and may affect the applicability of the report's conclusions.

No security assessment can identify every vulnerability or guarantee that a system is secure, error-free or protected against attack or loss. Only the components, assumptions, threat models and risk categories expressly identified as in scope were assessed. The report should be read in full, including its scope, exclusions, unresolved findings and remediation status. The client remains responsible for design, testing, remediation, deployment, operation, monitoring and incident response.

This report is a technical assessment, not a certification, attestation, investment recommendation, or legal, financial, regulatory or tax advice. It does not constitute an endorsement of the client, the assessed system or any related project.

This report was prepared for the client under a separate engagement agreement. If published, its public availability is for transparency and informational purposes only. To the fullest extent permitted by applicable law, publication does not create an auditor-client relationship, duty of care or right of reliance in favor of any third party. Other readers should conduct their own diligence and obtain appropriate professional advice.

Only the complete, unmodified report as issued by BailSec is authoritative. The signed engagement agreement exclusively governs the contractual rights and obligations between BailSec and the client, including warranties, remedies, liability limitations, indemnities and dispute resolution. Nothing in this notice excludes liability that cannot lawfully be excluded.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	1inch – Differential – Audit Report
Website	1inch.com
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/1inch/swap-vm/blob/abc9c94e62b9005085c3220cde46a94d96198673/src/instructions/Fee.sol
Resolution 1	https://github.com/1inch/swap-vm/blob/32c687c2b73101fc26549e48fa1ff8a4d73afbac/src/instructions/Fee.sol

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High						
Medium						
Low	2			2		
Informational	3			3		
Governance						
Total	5			5		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

This audit covers a change in the Fee.sol contract which changes the way how the fee is eventually pulled from the maker.

Previously, SwapVM protocol fees were taken and denominated in the token that the taker offers/pays to the maker. These fees were implemented by transferring that token from the maker to the fee recipient. Notably, this transfer happened *before* the payment from the taker to the maker is settled. The result is that for one-sided strategies in particular and strategies where the maker lacks a balance and/or allowance in general, it is not possible to fill such orders. This behavior was catalogued as Issue_53 in the previous Bailsec audit of the Aqua/SwapVM codebase, dated May 2026.

The goal of the change under review is to improve compatibility of SwapVM with these sorts of strategies and scenarios.

Protocol fee collection from the maker is now optional. If the maker doesn't have enough balance or has not granted sufficient allowance, the swap still succeeds, but the fee recipient receives nothing. The fee amount is nevertheless deducted from the taker and credited to the maker

last audited commit:

<https://github.com/linch/swap-vm/blob/b2daef83656b1ccbfc1c83d3df869c0ebb60badc/src/instructions/Fee.sol>

new commit:

<https://github.com/linch/swap-vm/blob/abc9c94e62b9005085c3220cde46a94d96198673/src/instructions/Fee.sol>

Only contract Fee is covered by this review. The changes can be seen below.

Determined changes:

- <https://www.diffchecker.com/fVBt0SES/>

Fee

Issue_01	Maker can abuse the optionality by never paying out protocol fees
Severity	Low
Description	Since the swap completes even if the protocol fee is not transferred out from the maker, a maker can divert the protocol fees to themselves by intentionally causing the fee transfer to always revert by setting the tokenIn approval for Aqua contract to 0
Recommendations	Re-work the mechanism to transfer fees directly from the taker to the fee recipient. Alternatively, do not validate/whitelist/index one-sided strategies.
Comments / Resolution	Acknowledged

Issue_02	Invalid recipient error name is not suitable for <code>_aquaProtocolFeeAmountInXD</code> instruction
Severity	Low
Description	In case the fee recipient address is set to 0, the error currently returned in the <code>_aquaProtocolFeeAmountInXD</code> instruction is <code>FeeDynamicProtocolInvalidRecipient</code> . This is copied over from the other protocol fee instructions where the nature of the instruction is actually dynamic unlike <code>_aquaProtocolFeeAmountInXD</code> instruction. Hence using the same error name is not suitable
Recommendations	Create a non-dynamic variant of the error
Comments / Resolution	Acknowledged

Issue_03	Unexpected compatibility with whitelist tokens
Severity	Informational
Description	In the previous revision of this functionality, there was no compatibility for strategies involving tokens with whitelists because attempting to take a fee in a token that has not whitelisted the recipient causes a revert. Now tokens with a whitelist will silently skip the protocol fee and are compatible with SwapVM/Aqua
Recommendations	Document and acknowledge this finding
Comments / Resolution	Acknowledged

Issue_04	<code>to</code> is not checked for 0 address in the <code>_protocolFeeAmountInXD</code> instruction
Severity	Informational
Description	All protocol fee instructions now enforce an <code>address[0]</code> check except the <code>_protocolFeeAmountInXD</code> instruction
Recommendations	Enforce <code>to != address[0]</code> in the <code>_protocolFeeAmountInXD</code> instruction as well
Comments / Resolution	Acknowledged

Issue_05	"Makers MUST NOT use backward jumps" only documented in comments
Severity	Informational
Description	The strategy construction requirement is documented only in comments in the Solidity source code. A requirement of this importance, the violation of which results in <code>quote()/swap()</code> inconsistency, should be documented in the protocol prose and enforced in an SDK
Recommendations	Document this requirement outside of the Solidity source
Comments / Resolution	Acknowledged